CI/CD Overview

Explore the fundamentals of Continuous Integration and Continuous Delivery on AWS, including key services like CodeBuild, CodeDeploy, and CodePipeline. Understand how to automate build and deployment pipelines, monitor applications, and apply these practices to AWS Developer Associate exam scenarios.

CI/CD is a modern approach to software development and delivery that enables continuous feedback and rapid iteration.

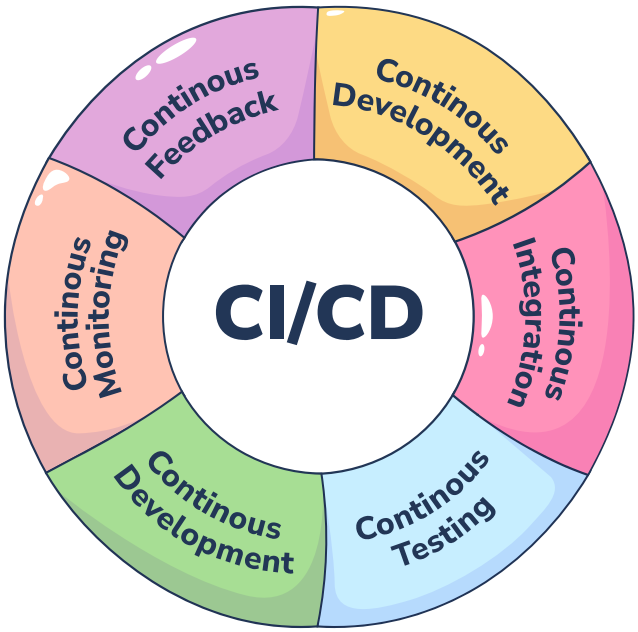
Continuous Integration (CI) is the practice of automatically integrating code from multiple developers into a shared repository several times a day. In AWS, CI ensures that every integration is verified by an automated build and test, typically using tools like AWS CodeBuild and GitHub.

Continuous Delivery (CD) builds on CI by automatically preparing and validating code changes for deployment to staging or production environments. This stage emphasizes test automation and environment parity to ensure release readiness.

Continuous Deployment, an extension of CD, goes a step further by automatically releasing every validated change into production. AWS supports this through services like AWS CodeDeploy, allowing seamless integration between testing and deployment.

These practices reduce manual errors, improve deployment speed, and make application behavior predictable across environments in AWS-based development.

Ask



CI/CD lifecycle

CI/CD benefits and AWS services

CI/CD pipelines empower developers to deliver software faster, reduce bugs in production, and respond quickly to customer needs. In the AWS context, these benefits are magnified through scalable and fully managed services.

The main AWS CI/CD services include:

- **AWS CodeBuild** to compile source code, run tests, and produce deployable artifacts.
- **AWS CodeDeploy** for automated application deployment to compute services like EC2, Lambda, and ECS.
- **AWS CodePipeline** as the orchestration service that connects all stages of the CI/CD process.

These services integrate seamlessly to form complete pipelines. For examp CodePipeline can be configured to trigger an ECS deployment whenever code is pushed to a GitHub repository, with CodeBuild compiling and testing the container image, and CodeDeploy handling updates to the ECS service. This

Ask

setup enables continuous delivery with built-in rollback and monitoring. This end-to-end pipeline supports fast feedback, safe deployments, and rollback capabilities.

Technical pillars of CI/CD on AWS

Several AWS services underpin the CI/CD mindset:

Infrastructure as code (IaC) allows developers to manage infrastructure the same way they manage application code using:

- AWS CloudFormation and AWS CDK to provision and update cloud resources declaratively.
- AWS SAM for serverless applications, integrating APIs, Lambda, and DynamoDB in reproducible templates.

Observability and Monitoring ensure applications are transparent and predictable:

- Amazon CloudWatch aggregates logs and metrics. A typical use case is monitoring Lambda function error rates and setting alarms when thresholds are exceeded.
- AWS X-Ray enables distributed tracing and visualizes the lifecycle of requests. For example, developers use it to trace API Gateway requests across microservices and identify bottlenecks.
- AWS Config tracks configuration changes across resources. A common use case is auditing IAM policy changes to ensure compliance with organizational security standards.

Automated Testing is embedded throughout CI/CD pipelines. Developers use mock services and test data to validate functionality before deployment, a integrate unit, integration, and end-to-end tests in CodeBuild or third-party tools.

Ask

CI/CD and DevOps on the DVA-C02 Exam

The AWS Certified Developer – Associate exam emphasizes understanding and using already existing CI/CD pipelines to handle code changes rather than designing them from scratch.

Candidates should expect scenario-based questions testing their knowledge of:

- Triggering deployments via code commits (e.g., to Lambda or ECS). A common use case is automatically deploying a new version of a Lambda function whenever developers push code to a GitHub source repository.
- Using Lambda aliases to manage staging environments. For example, developers use aliases like 'dev', 'test', and 'prod' to route traffic to specific function versions during phased rollouts.
- Implementing deployment strategies such as blue/green and canary using CodeDeploy. These approaches help minimize risk by shifting only a portion of traffic to a new version before full rollout.
- Managing pipeline stages and approvals in CodePipeline. This is often used to insert manual approval steps before promoting builds to production, ensuring greater control over deployments.

While pipeline architecture design is out of scope, candidates must be prof in automating and troubleshooting the deployment process using AWS Dev Tools.

Ask

As part of the exam, it is important to understand how this can be done programmatically. Here's some sample Python code given below that creates a basic AWS CodePipeline that automatically deploys a Lambda function when code is pushed to a repository.

```
1  from aws_cdk import (
2      aws_codepipeline as codepipeline,
3      aws_codepipeline_actions as actions,
4      aws_codebuild as codebuild,
5      aws_lambda as _lambda,
6      aws_iam as iam,
7      aws_codestarconnections as codestar,
8      Stack
9  )
10 from constructs import Construct
11
12 class PipelineStack(Stack):
13     def __init__(self, scope: Construct, construct_id: str, **kwargs):
14         super().__init__(scope, construct_id, **kwargs)
15
16         # Create a CodeStar connection to GitHub
17         github_connection = codestar.CfnConnection(
18             self, "GitHubConnection",
19             connection_name="MyGitHubConnection",
20             provider_type="GitHub"
21         )
22
23         lambda_func = _lambda.Function(
24             self, "LambdaFunction",
25             code=_lambda.Code.from_asset("lambda_code"),
26             handler="index.handler"
```

Python code to create a basic CodePipeline pipeline

Here's a line-by-line explanation of the code:

- **Lines 1–9:** Import AWS CDK libraries needed for pipeline, Lambda, CodeBuild, and GitHub integration.
- **Lines 12–53:** Define a new CDK stack class named `PipelineStack`.

Ask

- **Lines 17–21:** Set up a CodeStar connection resource to securely connect with a GitHub repository
- **Lines 23–28:** Define a Lambda function that will be deployed, using source code from a local folder.
- **Line 30:** Create a CodeBuild project to build and test the Lambda code.
- **Line 32:** Initialize the CodePipeline.
- **Line 33:** Define an artifact placeholder for the output of the source stage.
- **Lines 35–44:** Add a source stage that uses CodeStar Connections to pull code from a GitHub repository.
- **Lines 46–52:** Add a build stage that uses CodeBuild to process the source code and prepare it for deployment.

Conclusion

CI/CD practices are core to modern, agile software development in the cloud. On AWS, these paradigms are supported by a rich ecosystem of managed services that streamline code integration, testing, and deployment.

As developers, embracing CI/CD enables us to deliver features faster, recover from failures efficiently, and continuously improve our applications.

Ask